

DEVOPS MASTER COURSE

Volume 12D

Google Cloud Enterprise Engineering

*Organization • IAM • Shared VPC • Compute • Data • Security • Reliability •
Operations • FinOps*

Chapters 31-40 | 2026 Edition

Beginner to Enterprise Cloud Architecture and Operations

About This Part

Part D develops provider-depth in Google Cloud. It uses the organization-folder-project hierarchy, keyless identity, Shared VPC, managed compute and data services, Security Command Center, the Operations suite and billing exports to build an enterprise foundation.

2026 platform note: The 2026 edition follows Google Cloud landing-zone design guidance, enterprise foundation blueprints, Workload Identity Federation, Shared VPC and organization-policy controls rather than persistent service-account keys.

Volume 12 Roadmap

Part	Coverage	Status
12A	Multi-cloud architecture and landing-zone foundations	Completed
12B	AWS enterprise cloud engineering	Course sequence
12C	Azure enterprise cloud engineering	Course sequence
12D	Google Cloud enterprise engineering	Completed in this package
12E	Hybrid and multi-cloud operations, migration, DR, security and FinOps	Course sequence
12F	Enterprise projects, certifications, interviews and final capstone	Course sequence

Scope, Safety and Cost Controls

- Use disposable sandboxes, free tiers, local simulations or approved training subscriptions. Configure budgets and expiry before creating resources.
- Never use production identities, customer data, employer account structures, internal IP ranges or confidential contracts in course artifacts.
- Cloud services, prices, limits, recommendations and certifications change. Verify official documentation before production use or exam registration.
- Run failure, security and recovery tests only in approved nonproduction environments with explicit stop conditions.
- Destroy resources, revoke credentials, remove local state copies and verify billing inventory after every lab.

Learning Outcomes

- Explain and design the major architecture domains covered by Google Cloud Enterprise Engineering.
- Implement provider or cross-cloud controls through version-controlled automation.
- Test identity, network, security, reliability, operations and cost behavior.
- Troubleshoot failures using effective policy, route, service and telemetry evidence.
- Produce sanitized architecture, runbook, laboratory and interview portfolio artifacts.

Table of Contents

- Chapter 31 Google Cloud Organization, Folders, Projects and Landing Zones
- Chapter 32 Google Cloud IAM, Federation and Organization Policy
- Chapter 33 Google Cloud Networking, Shared VPC and Hybrid Connectivity
- Chapter 34 Google Cloud Compute, Serverless and GKE Architecture
- Chapter 35 Google Cloud Storage, Databases and Data Protection
- Chapter 36 Google Cloud Security Command Center and Data Perimeters
- Chapter 37 Google Cloud Reliability, Global Load Balancing and Disaster Recovery
- Chapter 38 Google Cloud Operations Suite and OpenTelemetry
- Chapter 39 Google Cloud FinOps, Billing Exports and Resource Governance
- Chapter 40 Google Cloud Enterprise Foundation Capstone
- Google Cloud Enterprise Architecture and Operations Reference
- Completion Checklist
- Additional Portfolio Projects
- 30-Day Practice Plan
- Glossary
- Official Documentation Index

Chapter 31: Google Cloud Organization, Folders, Projects and Landing Zones

Designing a resource hierarchy and foundation blueprint

Why this chapter matters: Google Cloud hierarchy determines inherited IAM, organization policies, billing and workload isolation. A durable design uses projects as strong lifecycle boundaries and centralizes foundation capabilities.

Learning Objectives

- Explain the architecture, responsibility boundaries and major trade-offs of organization, folders and project hierarchy.
- Design landing-zone design and enterprise foundation blueprint for enterprise scale, least privilege and bounded failure.
- Implement project factory and lifecycle automation through version-controlled automation and policy checks.
- Troubleshoot organization policy and centralized evidence using provider-native telemetry and repeatable evidence.
- Deliver a sanitized portfolio artifact for design and test a google cloud landing-zone hierarchy.

Core Concepts

Organization, folders and project hierarchy

Organization, folders and project hierarchy is a design boundary, not merely a console feature. In Google Cloud Organization, Folders, Projects and Landing Zones, the team must document ownership, trust assumptions, dependency direction, failure impact and the evidence used to prove the control. The recommended implementation uses Google Cloud organization, Folders, Projects, Organization Policy Service only where they satisfy an explicit requirement; equivalent services are not automatically interchangeable.

Landing-zone design and enterprise foundation blueprint

A production implementation of Landing-zone design and enterprise foundation blueprint separates the management plane from workload execution and constrains blast radius. Architecture reviews should evaluate quotas, regional behavior, identity propagation, private connectivity, encryption, logging and recovery. The operational contract must state who changes the control, who monitors it and how an emergency exception expires.

Project factory and lifecycle automation

For Project factory and lifecycle automation, repeatability matters more than a one-time successful deployment. Infrastructure as code, policy tests, deployment verification, drift detection and rollback evidence should be retained together. A design is incomplete when it works only for the original engineer or depends on undocumented console steps.

Organization policy and centralized evidence

The strongest validation for Organization policy and centralized evidence combines an allowed test, a denied test and a failure test. Teams should prove normal workload behavior, demonstrate that prohibited behavior is blocked and exercise recovery under a bounded fault. Metrics, logs, audit events and cost records become part of the acceptance package.

Service and Capability Map

Capability	Service or pattern	Enterprise use
Root	Google Cloud organization	Anchor enterprise resources and policy.
Hierarchy	Folders	Group platform, production, nonproduction and business units.

Capability	Service or pattern	Enterprise use
Workload boundary	Projects	Separate APIs, quotas, billing, IAM and lifecycle.
Guardrails	Organization Policy Service	Constrain resource configuration across hierarchy.
Vending	Project factory	Create standardized projects and baseline services.
Evidence	Cloud Audit Logs / log sinks	Centralize immutable administrative evidence.

Architecture and Operating Decisions

- Use Google Cloud managed services only after recording availability, security, portability and cost consequences.
- Separate production, nonproduction, security and shared services with strong administrative and billing boundaries.
- Prefer short-lived workload identity, private service access and centralized audit evidence over persistent credentials and public administration.
- Automate project factory and lifecycle automation with reviewed plans, policy tests, staged rollout and post-deployment verification.
- Define recovery objectives, quotas, ownership and decommissioning before the service becomes business critical.

Reference Configuration or Decision Pattern

```

organization
├── common
│   ├── logging
│   ├── security
│   └── networking
├── production
├── nonproduction
├── sandbox
└── decommissioned
project_request:
  required: [owner, billing_account, environment, region, data_classification, expiry]
```

Interpretation: Treat the pattern as a starting point. Replace placeholders with measured requirements, provider limits and approved organizational standards.

Production Readiness Checklist

- ☐ Architecture diagrams and responsibility boundaries for google cloud organization, folders, projects and landing zones are current and version controlled.
- ☐ Identity uses groups, roles, workload federation and privileged elevation with no unnecessary standing administrator access.
- ☐ Network paths, DNS behavior, encryption, audit logging and service limits are tested before production use.
- ☐ Changes are delivered through IaC with validation, policy, approval, rollback and drift detection.
- ☐ Recovery, cost allocation, ownership metadata and teardown have measurable evidence.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment succeeds but service is unreachable	Route, firewall, DNS or private-endpoint path is incomplete	Trace the path hop by hop; compare route, policy, DNS and endpoint state; then run an explicit reachability test.	Successful synthetic request and flow/audit evidence
Authorization is broader than intended	Inherited role, wildcard permission or unmanaged credential	Inspect effective permissions, credential age and access logs; replace broad grants with scoped roles and short-lived identity.	Effective-access review and denied-action test

Symptom	Likely cause	Corrective action	Validation evidence
Regional failure breaks the workload	State, dependency or failover mechanism remained single-region	Map stateful dependencies, restore order and traffic controls; exercise failover and fallback against measured RTO/RPO.	Timed recovery log with data-integrity checks
Cost rises without traffic growth	Idle capacity, data transfer, logging volume or commitment mismatch	Break down spend by owner, service, region and usage driver; validate rightsizing or commitment changes against SLOs.	Cost delta plus performance and reliability validation

Hands-on Lab 31: Design and Test a Google Cloud Landing-Zone Hierarchy

Prerequisites

- Disposable provider sandbox or a local simulation.
- Terraform or provider CLI, Git and a diagramming tool.
- Synthetic identities and data; no production credentials or confidential information.

Implementation

1. Create the architecture and responsibility model for google cloud organization, folders, projects and landing zones using fictional enterprise requirements.
2. Implement the smallest safe sandbox using Google Cloud organization, Folders, Projects; use free or low-cost options where available.
3. Apply identity, network, encryption, logging, metadata and budget controls through IaC or an equivalent version-controlled simulation.
4. Run one approved behavior test and one denied behavior test; save the commands, timestamps and resulting telemetry.
5. Inject a bounded configuration or dependency failure, follow the runbook, recover and record measured recovery time.
6. Destroy or quarantine the lab, revoke temporary access and verify that no unexpected billable resources remain.

Validation Evidence

- Version-controlled architecture, IaC and decision record.
- Allowed, denied and failure-test outputs with timestamps.
- Cost, recovery, ownership and teardown verification.

Cleanup

- Destroy test resources in dependency order and verify the provider inventory and billing view.
- Revoke temporary identities, remove local state copies and sanitize all portfolio evidence.

Interview Questions and Model Answers

1. What is the most important design principle in Google Cloud Organization, Folders, Projects and Landing Zones?

Make responsibility and failure boundaries explicit, then automate controls and prove them with normal, denied and recovery tests rather than trusting a successful deployment alone.

2. How would you choose between the main Google Cloud services in this domain?

Start with workload requirements for latency, state, scale, compliance, team capability, recovery and cost. Select the managed service whose operating model fits those requirements and document the consequences in an ADR.

3. What evidence would you require before production approval?

Architecture and threat models, reviewed IaC plan, effective-access results, network and encryption tests, known audit events, SLO and recovery validation, ownership metadata, budget routing and rollback evidence.

4. How do you reduce provider lock-in without creating a lowest-common-denominator platform?

Preserve portable source, data, interfaces and deployment artifacts where a credible exit scenario requires them, while still using provider-specific capabilities when their measurable value exceeds migration and dependency risk.

Chapter Summary

- Organization, folders and project hierarchy defines a key architecture boundary.
- Landing-zone design and enterprise foundation blueprint must be automated and least-privileged.
- Project factory and lifecycle automation requires policy, telemetry and recovery evidence.
- Organization policy and centralized evidence should be tested before production and after material change.

Chapter 32: Google Cloud IAM, Federation and Organization Policy

Authorizing workforce and workloads without service-account key sprawl

Why this chapter matters: Google Cloud permissions are powerful and frequently inherited. Workforce federation, groups, service accounts, Workload Identity Federation and organization policies must be designed as one control system.

Learning Objectives

- Explain the architecture, responsibility boundaries and major trade-offs of cloud identity or external workforce federation.
- Design iam roles, groups and conditions for enterprise scale, least privilege and bounded failure.
- Implement service accounts and workload identity federation through version-controlled automation and policy checks.
- Troubleshoot organization policies and privileged access using provider-native telemetry and repeatable evidence.
- Deliver a sanitized portfolio artifact for implement google cloud federation and keyless workload access.

Core Concepts

Cloud Identity or external workforce federation

Cloud Identity or external workforce federation is a design boundary, not merely a console feature. In Google Cloud IAM, Federation and Organization Policy, the team must document ownership, trust assumptions, dependency direction, failure impact and the evidence used to prove the control. The recommended implementation uses Cloud Identity / Workforce Identity Federation, Cloud IAM, Workload Identity Federation, Workload Identity Federation for GKE only where they satisfy an explicit requirement; equivalent services are not automatically interchangeable.

IAM roles, groups and conditions

A production implementation of IAM roles, groups and conditions separates the management plane from workload execution and constrains blast radius. Architecture reviews should evaluate quotas, regional behavior, identity propagation, private connectivity, encryption, logging and recovery. The operational contract must state who changes the control, who monitors it and how an emergency exception expires.

Service accounts and Workload Identity Federation

For Service accounts and Workload Identity Federation, repeatability matters more than a one-time successful deployment. Infrastructure as code, policy tests, deployment verification, drift detection and rollback evidence should be retained together. A design is incomplete when it works only for the original engineer or depends on undocumented console steps.

Organization policies and privileged access

The strongest validation for Organization policies and privileged access combines an allowed test, a denied test and a failure test. Teams should prove normal workload behavior, demonstrate that prohibited behavior is blocked and exercise recovery under a bounded fault. Metrics, logs, audit events and cost records become part of the acceptance package.

Service and Capability Map

Capability	Service or pattern	Enterprise use
Workforce	Cloud Identity / Workforce Identity Federation	Authenticate enterprise users without duplicate passwords.
Authorization	Cloud IAM	Grant predefined or custom roles at controlled scope.

Capability	Service or pattern	Enterprise use
Workload identity	Workload Identity Federation	Exchange external workload identity for short-lived Google credentials.
GKE identity	Workload Identity Federation for GKE	Bind Kubernetes service accounts to Google permissions.
Guardrail	Organization Policy Service	Restrict allowed configuration and locations.
Secrets	Secret Manager	Store and rotate application secrets.

Architecture and Operating Decisions

- Use Google Cloud managed services only after recording availability, security, portability and cost consequences.
- Separate production, nonproduction, security and shared services with strong administrative and billing boundaries.
- Prefer short-lived workload identity, private service access and centralized audit evidence over persistent credentials and public administration.
- Automate service accounts and workload identity federation with reviewed plans, policy tests, staged rollout and post-deployment verification.
- Define recovery objectives, quotas, ownership and decommissioning before the service becomes business critical.

Reference Configuration or Decision Pattern

```
identity_standard:
  workforce: external_idp_to_workforce_pool
  roles: groups_plus_predefined_roles
  workloads: workload_identity_federation
  service_account_keys: prohibited_by_policy
  privileged_access: time_bound_and_approved
  conditions: use_for_resource_and_time_constraints
  validation: [policy_analyzer, denied_action, audit_log, token_claims]
```

Interpretation: Treat the pattern as a starting point. Replace placeholders with measured requirements, provider limits and approved organizational standards.

Production Readiness Checklist

- ☐ Architecture diagrams and responsibility boundaries for google cloud iam, federation and organization policy are current and version controlled.
- ☐ Identity uses groups, roles, workload federation and privileged elevation with no unnecessary standing administrator access.
- ☐ Network paths, DNS behavior, encryption, audit logging and service limits are tested before production use.
- ☐ Changes are delivered through IaC with validation, policy, approval, rollback and drift detection.
- ☐ Recovery, cost allocation, ownership metadata and teardown have measurable evidence.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment succeeds but service is unreachable	Route, firewall, DNS or private-endpoint path is incomplete	Trace the path hop by hop; compare route, policy, DNS and endpoint state; then run an explicit reachability test.	Successful synthetic request and flow/audit evidence
Authorization is broader than intended	Inherited role, wildcard permission or unmanaged credential	Inspect effective permissions, credential age and access logs; replace broad grants with scoped roles and short-lived identity.	Effective-access review and denied-action test
Regional failure breaks the workload	State, dependency or failover mechanism remained single-region	Map stateful dependencies, restore order and traffic controls; exercise failover and fallback against measured RTO/RPO.	Timed recovery log with data-integrity checks

Symptom	Likely cause	Corrective action	Validation evidence
Cost rises without traffic growth	Idle capacity, data transfer, logging volume or commitment mismatch	Break down spend by owner, service, region and usage driver; validate rightsizing or commitment changes against SLOs.	Cost delta plus performance and reliability validation

Hands-on Lab 32: Implement Google Cloud Federation and Keyless Workload Access

Prerequisites

- Disposable provider sandbox or a local simulation.
- Terraform or provider CLI, Git and a diagramming tool.
- Synthetic identities and data; no production credentials or confidential information.

Implementation

1. Create the architecture and responsibility model for google cloud iam, federation and organization policy using fictional enterprise requirements.
2. Implement the smallest safe sandbox using Cloud Identity / Workforce Identity Federation, Cloud IAM, Workload Identity Federation; use free or low-cost options where available.
3. Apply identity, network, encryption, logging, metadata and budget controls through IaC or an equivalent version-controlled simulation.
4. Run one approved behavior test and one denied behavior test; save the commands, timestamps and resulting telemetry.
5. Inject a bounded configuration or dependency failure, follow the runbook, recover and record measured recovery time.
6. Destroy or quarantine the lab, revoke temporary access and verify that no unexpected billable resources remain.

Validation Evidence

- Version-controlled architecture, IaC and decision record.
- Allowed, denied and failure-test outputs with timestamps.
- Cost, recovery, ownership and teardown verification.

Cleanup

- Destroy test resources in dependency order and verify the provider inventory and billing view.
- Revoke temporary identities, remove local state copies and sanitize all portfolio evidence.

Interview Questions and Model Answers

1. What is the most important design principle in Google Cloud IAM, Federation and Organization Policy?

Make responsibility and failure boundaries explicit, then automate controls and prove them with normal, denied and recovery tests rather than trusting a successful deployment alone.

2. How would you choose between the main Google Cloud services in this domain?

Start with workload requirements for latency, state, scale, compliance, team capability, recovery and cost. Select the managed service whose operating model fits those requirements and document the consequences in an ADR.

3. What evidence would you require before production approval?

Architecture and threat models, reviewed IaC plan, effective-access results, network and encryption tests, known audit events, SLO and recovery validation, ownership metadata, budget routing and rollback evidence.

4. How do you reduce provider lock-in without creating a lowest-common-denominator platform?

Preserve portable source, data, interfaces and deployment artifacts where a credible exit scenario requires them, while still using provider-specific capabilities when their measurable value exceeds migration and dependency risk.

Chapter Summary

- Cloud Identity or external workforce federation defines a key architecture boundary.
- IAM roles, groups and conditions must be automated and least-privileged.
- Service accounts and Workload Identity Federation requires policy, telemetry and recovery evidence.
- Organization policies and privileged access should be tested before production and after material change.

Chapter 33: Google Cloud Networking, Shared VPC and Hybrid Connectivity

Building global VPC architecture with private service access

Why this chapter matters: Google Cloud VPCs are global while subnets are regional. Enterprise design must account for Shared VPC ownership, hybrid routing, DNS, private access and centralized security.

Learning Objectives

- Explain the architecture, responsibility boundaries and major trade-offs of shared vpc and centralized network ownership.
- Design cloud router, network connectivity center and hybrid routing for enterprise scale, least privilege and bounded failure.
- Implement cloud interconnect and ha vpn through version-controlled automation and policy checks.
- Troubleshoot private service connect, cloud dns and load balancing using provider-native telemetry and repeatable evidence.
- Deliver a sanitized portfolio artifact for build a shared vpc with private service connectivity.

Core Concepts

Shared VPC and centralized network ownership

Shared VPC and centralized network ownership is a design boundary, not merely a console feature. In Google Cloud Networking, Shared VPC and Hybrid Connectivity, the team must document ownership, trust assumptions, dependency direction, failure impact and the evidence used to prove the control. The recommended implementation uses Shared VPC, Cloud Router, Network Connectivity Center, Cloud Interconnect / HA VPN only where they satisfy an explicit requirement; equivalent services are not automatically interchangeable.

Cloud Router, Network Connectivity Center and hybrid routing

A production implementation of Cloud Router, Network Connectivity Center and hybrid routing separates the management plane from workload execution and constrains blast radius. Architecture reviews should evaluate quotas, regional behavior, identity propagation, private connectivity, encryption, logging and recovery. The operational contract must state who changes the control, who monitors it and how an emergency exception expires.

Cloud Interconnect and HA VPN

For Cloud Interconnect and HA VPN, repeatability matters more than a one-time successful deployment. Infrastructure as code, policy tests, deployment verification, drift detection and rollback evidence should be retained together. A design is incomplete when it works only for the original engineer or depends on undocumented console steps.

Private Service Connect, Cloud DNS and load balancing

The strongest validation for Private Service Connect, Cloud DNS and load balancing combines an allowed test, a denied test and a failure test. Teams should prove normal workload behavior, demonstrate that prohibited behavior is blocked and exercise recovery under a bounded fault. Metrics, logs, audit events and cost records become part of the acceptance package.

Service and Capability Map

Capability	Service or pattern	Enterprise use
Enterprise network	Shared VPC	Centralize network ownership while service projects host workloads.

Capability	Service or pattern	Enterprise use
Dynamic routing	Cloud Router	Exchange routes using BGP for hybrid services.
Connectivity hub	Network Connectivity Center	Coordinate spoke connectivity across networks and sites.
Hybrid	Cloud Interconnect / HA VPN	Provide private or encrypted on-premises connectivity.
Private services	Private Service Connect	Publish and consume services privately.
DNS	Cloud DNS	Operate managed public, private and forwarding zones.

Architecture and Operating Decisions

- Use Google Cloud managed services only after recording availability, security, portability and cost consequences.
- Separate production, nonproduction, security and shared services with strong administrative and billing boundaries.
- Prefer short-lived workload identity, private service access and centralized audit evidence over persistent credentials and public administration.
- Automate cloud interconnect and ha vpn with reviewed plans, policy tests, staged rollout and post-deployment verification.
- Define recovery objectives, quotas, ownership and decommissioning before the service becomes business critical.

Reference Configuration or Decision Pattern

```
network:
  vpc_model: shared_vpc_per_environment_or_trust_domain
  routing: global_dynamic_where_required
  hybrid: redundant_interconnect_plus_ha_vpn
  service_access: private_service_connect
  dns: cloud_dns_forwarding_and_peering
  egress: centralized_policy_and_nat
  validation: [connectivity_tests, flow_logs, route_inventory, dns_tests]
```

Interpretation: Treat the pattern as a starting point. Replace placeholders with measured requirements, provider limits and approved organizational standards.

Production Readiness Checklist

- ☐ Architecture diagrams and responsibility boundaries for google cloud networking, shared vpc and hybrid connectivity are current and version controlled.
- ☐ Identity uses groups, roles, workload federation and privileged elevation with no unnecessary standing administrator access.
- ☐ Network paths, DNS behavior, encryption, audit logging and service limits are tested before production use.
- ☐ Changes are delivered through IaC with validation, policy, approval, rollback and drift detection.
- ☐ Recovery, cost allocation, ownership metadata and teardown have measurable evidence.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment succeeds but service is unreachable	Route, firewall, DNS or private-endpoint path is incomplete	Trace the path hop by hop; compare route, policy, DNS and endpoint state; then run an explicit reachability test.	Successful synthetic request and flow/audit evidence
Authorization is broader than intended	Inherited role, wildcard permission or unmanaged credential	Inspect effective permissions, credential age and access logs; replace broad grants with scoped roles and short-lived identity.	Effective-access review and denied-action test

Symptom	Likely cause	Corrective action	Validation evidence
Regional failure breaks the workload	State, dependency or failover mechanism remained single-region	Map stateful dependencies, restore order and traffic controls; exercise failover and fallback against measured RTO/RPO.	Timed recovery log with data-integrity checks
Cost rises without traffic growth	Idle capacity, data transfer, logging volume or commitment mismatch	Break down spend by owner, service, region and usage driver; validate rightsizing or commitment changes against SLOs.	Cost delta plus performance and reliability validation

Hands-on Lab 33: Build a Shared VPC with Private Service Connectivity

Prerequisites

- Disposable provider sandbox or a local simulation.
- Terraform or provider CLI, Git and a diagramming tool.
- Synthetic identities and data; no production credentials or confidential information.

Implementation

1. Create the architecture and responsibility model for google cloud networking, shared vpc and hybrid connectivity using fictional enterprise requirements.
2. Implement the smallest safe sandbox using Shared VPC, Cloud Router, Network Connectivity Center; use free or low-cost options where available.
3. Apply identity, network, encryption, logging, metadata and budget controls through IaC or an equivalent version-controlled simulation.
4. Run one approved behavior test and one denied behavior test; save the commands, timestamps and resulting telemetry.
5. Inject a bounded configuration or dependency failure, follow the runbook, recover and record measured recovery time.
6. Destroy or quarantine the lab, revoke temporary access and verify that no unexpected billable resources remain.

Validation Evidence

- Version-controlled architecture, IaC and decision record.
- Allowed, denied and failure-test outputs with timestamps.
- Cost, recovery, ownership and teardown verification.

Cleanup

- Destroy test resources in dependency order and verify the provider inventory and billing view.
- Revoke temporary identities, remove local state copies and sanitize all portfolio evidence.

Interview Questions and Model Answers

1. What is the most important design principle in Google Cloud Networking, Shared VPC and Hybrid Connectivity?

Make responsibility and failure boundaries explicit, then automate controls and prove them with normal, denied and recovery tests rather than trusting a successful deployment alone.

2. How would you choose between the main Google Cloud services in this domain?

Start with workload requirements for latency, state, scale, compliance, team capability, recovery and cost. Select the managed service whose operating model fits those requirements and document the consequences in an ADR.

3. What evidence would you require before production approval?

Architecture and threat models, reviewed IaC plan, effective-access results, network and encryption tests, known audit events, SLO and recovery validation, ownership metadata, budget routing and rollback evidence.

4. How do you reduce provider lock-in without creating a lowest-common-denominator platform?

Preserve portable source, data, interfaces and deployment artifacts where a credible exit scenario requires them, while still using provider-specific capabilities when their measurable value exceeds migration and dependency risk.

Chapter Summary

- Shared VPC and centralized network ownership defines a key architecture boundary.
- Cloud Router, Network Connectivity Center and hybrid routing must be automated and least-privileged.
- Cloud Interconnect and HA VPN requires policy, telemetry and recovery evidence.
- Private Service Connect, Cloud DNS and load balancing should be tested before production and after material change.

Chapter 34: Google Cloud Compute, Serverless and GKE Architecture

Choosing Compute Engine, Cloud Run, functions and Kubernetes

Why this chapter matters: Google Cloud compute placement should reflect runtime control, scaling, networking, event model and team operating capability. Managed services reduce some operational work but add platform-specific constraints.

Learning Objectives

- Explain the architecture, responsibility boundaries and major trade-offs of compute engine and managed instance groups.
- Design cloud run and event-driven serverless for enterprise scale, least privilege and bounded failure.
- Implement google kubernetes engine architecture through version-controlled automation and policy checks.
- Troubleshoot images, autoscaling and safe delivery using provider-native telemetry and repeatable evidence.
- Deliver a sanitized portfolio artifact for deploy and compare google cloud compute patterns.

Core Concepts

Compute Engine and managed instance groups

Compute Engine and managed instance groups is a design boundary, not merely a console feature. In Google Cloud Compute, Serverless and GKE Architecture, the team must document ownership, trust assumptions, dependency direction, failure impact and the evidence used to prove the control. The recommended implementation uses Compute Engine / managed instance groups, Cloud Run, Cloud Run functions, Google Kubernetes Engine only where they satisfy an explicit requirement; equivalent services are not automatically interchangeable.

Cloud Run and event-driven serverless

A production implementation of Cloud Run and event-driven serverless separates the management plane from workload execution and constrains blast radius. Architecture reviews should evaluate quotas, regional behavior, identity propagation, private connectivity, encryption, logging and recovery. The operational contract must state who changes the control, who monitors it and how an emergency exception expires.

Google Kubernetes Engine architecture

For Google Kubernetes Engine architecture, repeatability matters more than a one-time successful deployment. Infrastructure as code, policy tests, deployment verification, drift detection and rollback evidence should be retained together. A design is incomplete when it works only for the original engineer or depends on undocumented console steps.

Images, autoscaling and safe delivery

The strongest validation for Images, autoscaling and safe delivery combines an allowed test, a denied test and a failure test. Teams should prove normal workload behavior, demonstrate that prohibited behavior is blocked and exercise recovery under a bounded fault. Metrics, logs, audit events and cost records become part of the acceptance package.

Service and Capability Map

Capability	Service or pattern	Enterprise use
Virtual machines	Compute Engine / managed instance groups	Run customized and legacy workloads.
Serverless containers	Cloud Run	Deploy stateless services and jobs with managed scaling.
Functions	Cloud Run functions	Run event-driven functions integrated with Google services.

Capability	Service or pattern	Enterprise use
Kubernetes	Google Kubernetes Engine	Operate Kubernetes workloads with managed capabilities.
Images	Artifact Registry / image pipelines	Store and govern container and language artifacts.
Autoscaling	MIG / GKE / Cloud Run autoscaling	Scale from measured demand with tested limits.

Architecture and Operating Decisions

- Use Google Cloud managed services only after recording availability, security, portability and cost consequences.
- Separate production, nonproduction, security and shared services with strong administrative and billing boundaries.
- Prefer short-lived workload identity, private service access and centralized audit evidence over persistent credentials and public administration.
- Automate google kubernetes engine architecture with reviewed plans, policy tests, staged rollout and post-deployment verification.
- Define recovery objectives, quotas, ownership and decommissioning before the service becomes business critical.

Reference Configuration or Decision Pattern

```
placement:
  custom_os_or_legacy: compute_engine_mig
  stateless_http_or_jobs: cloud_run
  event_functions: cloud_run_functions
  kubernetes_platform: gke
controls: [multi_zone, workload_identity, artifact_scanning, autoscaling_limits, rollback]
```

Interpretation: Treat the pattern as a starting point. Replace placeholders with measured requirements, provider limits and approved organizational standards.

Production Readiness Checklist

- ☐ Architecture diagrams and responsibility boundaries for google cloud compute, serverless and gke architecture are current and version controlled.
- ☐ Identity uses groups, roles, workload federation and privileged elevation with no unnecessary standing administrator access.
- ☐ Network paths, DNS behavior, encryption, audit logging and service limits are tested before production use.
- ☐ Changes are delivered through IaC with validation, policy, approval, rollback and drift detection.
- ☐ Recovery, cost allocation, ownership metadata and teardown have measurable evidence.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment succeeds but service is unreachable	Route, firewall, DNS or private-endpoint path is incomplete	Trace the path hop by hop; compare route, policy, DNS and endpoint state; then run an explicit reachability test.	Successful synthetic request and flow/audit evidence
Authorization is broader than intended	Inherited role, wildcard permission or unmanaged credential	Inspect effective permissions, credential age and access logs; replace broad grants with scoped roles and short-lived identity.	Effective-access review and denied-action test
Regional failure breaks the workload	State, dependency or failover mechanism remained single-region	Map stateful dependencies, restore order and traffic controls; exercise failover and fallback against measured RTO/RPO.	Timed recovery log with data-integrity checks

Symptom	Likely cause	Corrective action	Validation evidence
Cost rises without traffic growth	Idle capacity, data transfer, logging volume or commitment mismatch	Break down spend by owner, service, region and usage driver; validate rightsizing or commitment changes against SLOs.	Cost delta plus performance and reliability validation

Hands-on Lab 34: Deploy and Compare Google Cloud Compute Patterns

Prerequisites

- Disposable provider sandbox or a local simulation.
- Terraform or provider CLI, Git and a diagramming tool.
- Synthetic identities and data; no production credentials or confidential information.

Implementation

1. Create the architecture and responsibility model for google cloud compute, serverless and gke architecture using fictional enterprise requirements.
2. Implement the smallest safe sandbox using Compute Engine / managed instance groups, Cloud Run, Cloud Run functions; use free or low-cost options where available.
3. Apply identity, network, encryption, logging, metadata and budget controls through IaC or an equivalent version-controlled simulation.
4. Run one approved behavior test and one denied behavior test; save the commands, timestamps and resulting telemetry.
5. Inject a bounded configuration or dependency failure, follow the runbook, recover and record measured recovery time.
6. Destroy or quarantine the lab, revoke temporary access and verify that no unexpected billable resources remain.

Validation Evidence

- Version-controlled architecture, IaC and decision record.
- Allowed, denied and failure-test outputs with timestamps.
- Cost, recovery, ownership and teardown verification.

Cleanup

- Destroy test resources in dependency order and verify the provider inventory and billing view.
- Revoke temporary identities, remove local state copies and sanitize all portfolio evidence.

Interview Questions and Model Answers

1. What is the most important design principle in Google Cloud Compute, Serverless and GKE Architecture?

Make responsibility and failure boundaries explicit, then automate controls and prove them with normal, denied and recovery tests rather than trusting a successful deployment alone.

2. How would you choose between the main Google Cloud services in this domain?

Start with workload requirements for latency, state, scale, compliance, team capability, recovery and cost. Select the managed service whose operating model fits those requirements and document the consequences in an ADR.

3. What evidence would you require before production approval?

Architecture and threat models, reviewed IaC plan, effective-access results, network and encryption tests, known audit events, SLO and recovery validation, ownership metadata, budget routing and rollback evidence.

4. How do you reduce provider lock-in without creating a lowest-common-denominator platform?

Preserve portable source, data, interfaces and deployment artifacts where a credible exit scenario requires them, while still using provider-specific capabilities when their measurable value exceeds migration and dependency risk.

Chapter Summary

- Compute Engine and managed instance groups defines a key architecture boundary.
- Cloud Run and event-driven serverless must be automated and least-privileged.
- Google Kubernetes Engine architecture requires policy, telemetry and recovery evidence.
- Images, autoscaling and safe delivery should be tested before production and after material change.

Chapter 35: Google Cloud Storage, Databases and Data Protection

Selecting data services for scale, consistency and recovery

Why this chapter matters: Google Cloud offers global and regional data services with different consistency, scaling and operational models. Data classification and recovery objectives must drive the choice.

Learning Objectives

- Explain the architecture, responsibility boundaries and major trade-offs of cloud storage lifecycle and redundancy.
- Design persistent disk and filestore for enterprise scale, least privilege and bounded failure.
- Implement cloud sql, spanner, bigtable and firestore through version-controlled automation and policy checks.
- Troubleshoot backup, replication and recovery validation using provider-native telemetry and repeatable evidence.
- Deliver a sanitized portfolio artifact for design and test google cloud data recovery.

Core Concepts

Cloud Storage lifecycle and redundancy

Cloud Storage lifecycle and redundancy is a design boundary, not merely a console feature. In Google Cloud Storage, Databases and Data Protection, the team must document ownership, trust assumptions, dependency direction, failure impact and the evidence used to prove the control. The recommended implementation uses Cloud Storage, Persistent Disk / Hyperdisk, Filestore, Cloud SQL / AlloyDB only where they satisfy an explicit requirement; equivalent services are not automatically interchangeable.

Persistent Disk and Filestore

A production implementation of Persistent Disk and Filestore separates the management plane from workload execution and constrains blast radius. Architecture reviews should evaluate quotas, regional behavior, identity propagation, private connectivity, encryption, logging and recovery. The operational contract must state who changes the control, who monitors it and how an emergency exception expires.

Cloud SQL, Spanner, Bigtable and Firestore

For Cloud SQL, Spanner, Bigtable and Firestore, repeatability matters more than a one-time successful deployment. Infrastructure as code, policy tests, deployment verification, drift detection and rollback evidence should be retained together. A design is incomplete when it works only for the original engineer or depends on undocumented console steps.

Backup, replication and recovery validation

The strongest validation for Backup, replication and recovery validation combines an allowed test, a denied test and a failure test. Teams should prove normal workload behavior, demonstrate that prohibited behavior is blocked and exercise recovery under a bounded fault. Metrics, logs, audit events and cost records become part of the acceptance package.

Service and Capability Map

Capability	Service or pattern	Enterprise use
Object	Cloud Storage	Store objects, archives and analytical datasets.
Block	Persistent Disk / Hyperdisk	Provide durable block storage for Compute Engine.
File	Filestore	Deliver managed NFS file services.

Capability	Service or pattern	Enterprise use
Relational	Cloud SQL / AlloyDB	Run managed relational databases.
Global relational	Spanner	Provide horizontally scalable relational consistency.
NoSQL	Bigtable / Firestore	Support wide-column or document access patterns.

Architecture and Operating Decisions

- Use Google Cloud managed services only after recording availability, security, portability and cost consequences.
- Separate production, nonproduction, security and shared services with strong administrative and billing boundaries.
- Prefer short-lived workload identity, private service access and centralized audit evidence over persistent credentials and public administration.
- Automate cloud sql, spanner, bigtable and firestore with reviewed plans, policy tests, staged rollout and post-deployment verification.
- Define recovery objectives, quotas, ownership and decommissioning before the service becomes business critical.

Reference Configuration or Decision Pattern

```
data_policy:
  classification: confidential
  locations: approved_regions_or_multi_regions
  public_access_prevention: enforced
  encryption: default_or_cmek_by_risk
  backups: scheduled_and_protected
  restore_test: quarterly
  export_and_exit: documented
```

Interpretation: Treat the pattern as a starting point. Replace placeholders with measured requirements, provider limits and approved organizational standards.

Production Readiness Checklist

- ☐ Architecture diagrams and responsibility boundaries for google cloud storage, databases and data protection are current and version controlled.
- ☐ Identity uses groups, roles, workload federation and privileged elevation with no unnecessary standing administrator access.
- ☐ Network paths, DNS behavior, encryption, audit logging and service limits are tested before production use.
- ☐ Changes are delivered through IaC with validation, policy, approval, rollback and drift detection.
- ☐ Recovery, cost allocation, ownership metadata and teardown have measurable evidence.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment succeeds but service is unreachable	Route, firewall, DNS or private-endpoint path is incomplete	Trace the path hop by hop; compare route, policy, DNS and endpoint state; then run an explicit reachability test.	Successful synthetic request and flow/audit evidence
Authorization is broader than intended	Inherited role, wildcard permission or unmanaged credential	Inspect effective permissions, credential age and access logs; replace broad grants with scoped roles and short-lived identity.	Effective-access review and denied-action test
Regional failure breaks the workload	State, dependency or failover mechanism remained single-region	Map stateful dependencies, restore order and traffic controls; exercise failover and fallback against measured RTO/RPO.	Timed recovery log with data-integrity checks

Symptom	Likely cause	Corrective action	Validation evidence
Cost rises without traffic growth	Idle capacity, data transfer, logging volume or commitment mismatch	Break down spend by owner, service, region and usage driver; validate rightsizing or commitment changes against SLOs.	Cost delta plus performance and reliability validation

Hands-on Lab 35: Design and Test Google Cloud Data Recovery

Prerequisites

- Disposable provider sandbox or a local simulation.
- Terraform or provider CLI, Git and a diagramming tool.
- Synthetic identities and data; no production credentials or confidential information.

Implementation

1. Create the architecture and responsibility model for google cloud storage, databases and data protection using fictional enterprise requirements.
2. Implement the smallest safe sandbox using Cloud Storage, Persistent Disk / Hyperdisk, Filestore; use free or low-cost options where available.
3. Apply identity, network, encryption, logging, metadata and budget controls through IaC or an equivalent version-controlled simulation.
4. Run one approved behavior test and one denied behavior test; save the commands, timestamps and resulting telemetry.
5. Inject a bounded configuration or dependency failure, follow the runbook, recover and record measured recovery time.
6. Destroy or quarantine the lab, revoke temporary access and verify that no unexpected billable resources remain.

Validation Evidence

- Version-controlled architecture, IaC and decision record.
- Allowed, denied and failure-test outputs with timestamps.
- Cost, recovery, ownership and teardown verification.

Cleanup

- Destroy test resources in dependency order and verify the provider inventory and billing view.
- Revoke temporary identities, remove local state copies and sanitize all portfolio evidence.

Interview Questions and Model Answers

1. What is the most important design principle in Google Cloud Storage, Databases and Data Protection?

Make responsibility and failure boundaries explicit, then automate controls and prove them with normal, denied and recovery tests rather than trusting a successful deployment alone.

2. How would you choose between the main Google Cloud services in this domain?

Start with workload requirements for latency, state, scale, compliance, team capability, recovery and cost. Select the managed service whose operating model fits those requirements and document the consequences in an ADR.

3. What evidence would you require before production approval?

Architecture and threat models, reviewed IaC plan, effective-access results, network and encryption tests, known audit events, SLO and recovery validation, ownership metadata, budget routing and rollback evidence.

4. How do you reduce provider lock-in without creating a lowest-common-denominator platform?

Preserve portable source, data, interfaces and deployment artifacts where a credible exit scenario requires them, while still using provider-specific capabilities when their measurable value exceeds migration and dependency risk.

Chapter Summary

- Cloud Storage lifecycle and redundancy defines a key architecture boundary.
- Persistent Disk and Filestore must be automated and least-privileged.
- Cloud SQL, Spanner, Bigtable and Firestore requires policy, telemetry and recovery evidence.
- Backup, replication and recovery validation should be tested before production and after material change.

Chapter 36: Google Cloud Security Command Center and Data Perimeters

Combining posture, threat detection, encryption and exfiltration controls

Why this chapter matters: Google Cloud security requires organization-level visibility and workload-level enforcement. VPC Service Controls, IAM, keys and network controls must be tested together to avoid both gaps and operational lockout.

Learning Objectives

- Explain the architecture, responsibility boundaries and major trade-offs of security command center posture and findings.
- Design cloud kms, secret manager and key governance for enterprise scale, least privilege and bounded failure.
- Implement vpc service controls and sensitive-data perimeters through version-controlled automation and policy checks.
- Troubleshoot cloud armor, ids and centralized response using provider-native telemetry and repeatable evidence.
- Deliver a sanitized portfolio artifact for build a google cloud security baseline and data perimeter test.

Core Concepts

Security Command Center posture and findings

Security Command Center posture and findings is a design boundary, not merely a console feature. In Google Cloud Security Command Center and Data Perimeters, the team must document ownership, trust assumptions, dependency direction, failure impact and the evidence used to prove the control. The recommended implementation uses Security Command Center, Cloud KMS / Cloud HSM, Secret Manager, VPC Service Controls only where they satisfy an explicit requirement; equivalent services are not automatically interchangeable.

Cloud KMS, Secret Manager and key governance

A production implementation of Cloud KMS, Secret Manager and key governance separates the management plane from workload execution and constrains blast radius. Architecture reviews should evaluate quotas, regional behavior, identity propagation, private connectivity, encryption, logging and recovery. The operational contract must state who changes the control, who monitors it and how an emergency exception expires.

VPC Service Controls and sensitive-data perimeters

For VPC Service Controls and sensitive-data perimeters, repeatability matters more than a one-time successful deployment. Infrastructure as code, policy tests, deployment verification, drift detection and rollback evidence should be retained together. A design is incomplete when it works only for the original engineer or depends on undocumented console steps.

Cloud Armor, IDS and centralized response

The strongest validation for Cloud Armor, IDS and centralized response combines an allowed test, a denied test and a failure test. Teams should prove normal workload behavior, demonstrate that prohibited behavior is blocked and exercise recovery under a bounded fault. Metrics, logs, audit events and cost records become part of the acceptance package.

Service and Capability Map

Capability	Service or pattern	Enterprise use
Posture and threats	Security Command Center	Aggregate misconfiguration, exposure and threat findings.
Keys	Cloud KMS / Cloud HSM	Control encryption keys and cryptographic operations.
Secrets	Secret Manager	Store and access secrets through IAM.

Capability	Service or pattern	Enterprise use
Data perimeter	VPC Service Controls	Reduce data exfiltration risk for supported services.
Edge	Cloud Armor	Protect internet-facing applications and APIs.
Network detection	Cloud IDS	Detect suspicious network activity.

Architecture and Operating Decisions

- Use Google Cloud managed services only after recording availability, security, portability and cost consequences.
- Separate production, nonproduction, security and shared services with strong administrative and billing boundaries.
- Prefer short-lived workload identity, private service access and centralized audit evidence over persistent credentials and public administration.
- Automate vpc service controls and sensitive-data perimeters with reviewed plans, policy tests, staged rollout and post-deployment verification.
- Define recovery objectives, quotas, ownership and decommissioning before the service becomes business critical.

Reference Configuration or Decision Pattern

```
security_baseline:
  scc: organization_level_activation
  audit_logs: central_aggregated_sink
  service_account_keys: disabled
  vpc_sc: applied_to_high_value_data_services
  kms: separation_of_key_admin_and_user
  response: finding_to_case_to_containment
  validation: [denied_exfiltration, audit_event, finding_routing, key_access_test]
```

Interpretation: Treat the pattern as a starting point. Replace placeholders with measured requirements, provider limits and approved organizational standards.

Production Readiness Checklist

- ☐ Architecture diagrams and responsibility boundaries for google cloud security command center and data perimeters are current and version controlled.
- ☐ Identity uses groups, roles, workload federation and privileged elevation with no unnecessary standing administrator access.
- ☐ Network paths, DNS behavior, encryption, audit logging and service limits are tested before production use.
- ☐ Changes are delivered through IaC with validation, policy, approval, rollback and drift detection.
- ☐ Recovery, cost allocation, ownership metadata and teardown have measurable evidence.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment succeeds but service is unreachable	Route, firewall, DNS or private-endpoint path is incomplete	Trace the path hop by hop; compare route, policy, DNS and endpoint state; then run an explicit reachability test.	Successful synthetic request and flow/audit evidence
Authorization is broader than intended	Inherited role, wildcard permission or unmanaged credential	Inspect effective permissions, credential age and access logs; replace broad grants with scoped roles and short-lived identity.	Effective-access review and denied-action test
Regional failure breaks the workload	State, dependency or failover mechanism remained single-region	Map stateful dependencies, restore order and traffic controls; exercise failover and fallback against measured RTO/RPO.	Timed recovery log with data-integrity checks
Cost rises without traffic growth	Idle capacity, data transfer, logging volume or commitment mismatch	Break down spend by owner, service, region and usage driver; validate rightsizing or commitment changes against SLOs.	Cost delta plus performance and reliability validation

Hands-on Lab 36: Build a Google Cloud Security Baseline and Data Perimeter Test

Prerequisites

- Disposable provider sandbox or a local simulation.
- Terraform or provider CLI, Git and a diagramming tool.
- Synthetic identities and data; no production credentials or confidential information.

Implementation

1. Create the architecture and responsibility model for google cloud security command center and data perimeters using fictional enterprise requirements.
2. Implement the smallest safe sandbox using Security Command Center, Cloud KMS / Cloud HSM, Secret Manager; use free or low-cost options where available.
3. Apply identity, network, encryption, logging, metadata and budget controls through IaC or an equivalent version-controlled simulation.
4. Run one approved behavior test and one denied behavior test; save the commands, timestamps and resulting telemetry.
5. Inject a bounded configuration or dependency failure, follow the runbook, recover and record measured recovery time.
6. Destroy or quarantine the lab, revoke temporary access and verify that no unexpected billable resources remain.

Validation Evidence

- Version-controlled architecture, IaC and decision record.
- Allowed, denied and failure-test outputs with timestamps.
- Cost, recovery, ownership and teardown verification.

Cleanup

- Destroy test resources in dependency order and verify the provider inventory and billing view.
- Revoke temporary identities, remove local state copies and sanitize all portfolio evidence.

Interview Questions and Model Answers

1. What is the most important design principle in Google Cloud Security Command Center and Data Perimeters?

Make responsibility and failure boundaries explicit, then automate controls and prove them with normal, denied and recovery tests rather than trusting a successful deployment alone.

2. How would you choose between the main Google Cloud services in this domain?

Start with workload requirements for latency, state, scale, compliance, team capability, recovery and cost. Select the managed service whose operating model fits those requirements and document the consequences in an ADR.

3. What evidence would you require before production approval?

Architecture and threat models, reviewed IaC plan, effective-access results, network and encryption tests, known audit events, SLO and recovery validation, ownership metadata, budget routing and rollback evidence.

4. How do you reduce provider lock-in without creating a lowest-common-denominator platform?

Preserve portable source, data, interfaces and deployment artifacts where a credible exit scenario requires them, while still using provider-specific capabilities when their measurable value exceeds migration and dependency risk.

Chapter Summary

- Security Command Center posture and findings defines a key architecture boundary.
- Cloud KMS, Secret Manager and key governance must be automated and least-privileged.
- VPC Service Controls and sensitive-data perimeters requires policy, telemetry and recovery evidence.
- Cloud Armor, IDS and centralized response should be tested before production and after material change.

Chapter 37: Google Cloud Reliability, Global Load Balancing and Disaster Recovery

Designing zonal, regional and multi-region recovery

Why this chapter matters: Google Cloud global infrastructure provides powerful routing options, but application state and dependencies still determine recoverability. Failover must be rehearsed and reversible.

Learning Objectives

- Explain the architecture, responsibility boundaries and major trade-offs of zonal and regional workload resilience.
- Design global load balancing and health checks for enterprise scale, least privilege and bounded failure.
- Implement cross-region data and backup patterns through version-controlled automation and policy checks.
- Troubleshoot recovery orchestration and failback using provider-native telemetry and repeatable evidence.
- Deliver a sanitized portfolio artifact for run a google cloud regional recovery exercise.

Core Concepts

Zonal and regional workload resilience

Zonal and regional workload resilience is a design boundary, not merely a console feature. In Google Cloud Reliability, Global Load Balancing and Disaster Recovery, the team must document ownership, trust assumptions, dependency direction, failure impact and the evidence used to prove the control. The recommended implementation uses Regional managed instance groups / GKE regional clusters, Cloud Load Balancing, Cloud DNS routing policies, Spanner / cross-region replicas / database replicas only where they satisfy an explicit requirement; equivalent services are not automatically interchangeable.

Global load balancing and health checks

A production implementation of Global load balancing and health checks separates the management plane from workload execution and constrains blast radius. Architecture reviews should evaluate quotas, regional behavior, identity propagation, private connectivity, encryption, logging and recovery. The operational contract must state who changes the control, who monitors it and how an emergency exception expires.

Cross-region data and backup patterns

For Cross-region data and backup patterns, repeatability matters more than a one-time successful deployment. Infrastructure as code, policy tests, deployment verification, drift detection and rollback evidence should be retained together. A design is incomplete when it works only for the original engineer or depends on undocumented console steps.

Recovery orchestration and failback

The strongest validation for Recovery orchestration and failback combines an allowed test, a denied test and a failure test. Teams should prove normal workload behavior, demonstrate that prohibited behavior is blocked and exercise recovery under a bounded fault. Metrics, logs, audit events and cost records become part of the acceptance package.

Service and Capability Map

Capability	Service or pattern	Enterprise use
Compute resilience	Regional managed instance groups / GKE regional clusters	Distribute workloads across zones.
Traffic	Cloud Load Balancing	Route global or regional traffic using health checks.
DNS	Cloud DNS routing policies	Provide DNS-based steering where appropriate.

Capability	Service or pattern	Enterprise use
Database continuity	Spanner / cross-region replicas / database replicas	Protect data according to service capability.
Backup	Backup and DR Service / snapshots	Protect and recover supported workloads.
Service continuity	Cloud Monitoring / Service Health	Detect workload and provider impairment.

Architecture and Operating Decisions

- Use Google Cloud managed services only after recording availability, security, portability and cost consequences.
- Separate production, nonproduction, security and shared services with strong administrative and billing boundaries.
- Prefer short-lived workload identity, private service access and centralized audit evidence over persistent credentials and public administration.
- Automate cross-region data and backup patterns with reviewed plans, policy tests, staged rollout and post-deployment verification.
- Define recovery objectives, quotas, ownership and decommissioning before the service becomes business critical.

Reference Configuration or Decision Pattern

```
recovery_tiers:
  critical: multi_region_with_global_load_balancing
  important: warm_secondary_and_replication
  standard: backup_restore
exercise:
  - block_primary_dependency
  - validate_secondary_capacity
  - promote_or_restore_data
  - shift_traffic
  - verify_integrity
  - fallback_controlled
```

Interpretation: Treat the pattern as a starting point. Replace placeholders with measured requirements, provider limits and approved organizational standards.

Production Readiness Checklist

- ☐ Architecture diagrams and responsibility boundaries for google cloud reliability, global load balancing and disaster recovery are current and version controlled.
- ☐ Identity uses groups, roles, workload federation and privileged elevation with no unnecessary standing administrator access.
- ☐ Network paths, DNS behavior, encryption, audit logging and service limits are tested before production use.
- ☐ Changes are delivered through IaC with validation, policy, approval, rollback and drift detection.
- ☐ Recovery, cost allocation, ownership metadata and teardown have measurable evidence.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment succeeds but service is unreachable	Route, firewall, DNS or private-endpoint path is incomplete	Trace the path hop by hop; compare route, policy, DNS and endpoint state; then run an explicit reachability test.	Successful synthetic request and flow/audit evidence
Authorization is broader than intended	Inherited role, wildcard permission or unmanaged credential	Inspect effective permissions, credential age and access logs; replace broad grants with scoped roles and short-lived identity.	Effective-access review and denied-action test
Regional failure breaks the workload	State, dependency or failover mechanism remained single-region	Map stateful dependencies, restore order and traffic controls; exercise failover and fallback against measured RTO/RPO.	Timed recovery log with data-integrity checks

Symptom	Likely cause	Corrective action	Validation evidence
Cost rises without traffic growth	Idle capacity, data transfer, logging volume or commitment mismatch	Break down spend by owner, service, region and usage driver; validate rightsizing or commitment changes against SLOs.	Cost delta plus performance and reliability validation

Hands-on Lab 37: Run a Google Cloud Regional Recovery Exercise

Prerequisites

- Disposable provider sandbox or a local simulation.
- Terraform or provider CLI, Git and a diagramming tool.
- Synthetic identities and data; no production credentials or confidential information.

Implementation

1. Create the architecture and responsibility model for google cloud reliability, global load balancing and disaster recovery using fictional enterprise requirements.
2. Implement the smallest safe sandbox using Regional managed instance groups / GKE regional clusters, Cloud Load Balancing, Cloud DNS routing policies; use free or low-cost options where available.
3. Apply identity, network, encryption, logging, metadata and budget controls through IaC or an equivalent version-controlled simulation.
4. Run one approved behavior test and one denied behavior test; save the commands, timestamps and resulting telemetry.
5. Inject a bounded configuration or dependency failure, follow the runbook, recover and record measured recovery time.
6. Destroy or quarantine the lab, revoke temporary access and verify that no unexpected billable resources remain.

Validation Evidence

- Version-controlled architecture, IaC and decision record.
- Allowed, denied and failure-test outputs with timestamps.
- Cost, recovery, ownership and teardown verification.

Cleanup

- Destroy test resources in dependency order and verify the provider inventory and billing view.
- Revoke temporary identities, remove local state copies and sanitize all portfolio evidence.

Interview Questions and Model Answers

1. What is the most important design principle in Google Cloud Reliability, Global Load Balancing and Disaster Recovery?

Make responsibility and failure boundaries explicit, then automate controls and prove them with normal, denied and recovery tests rather than trusting a successful deployment alone.

2. How would you choose between the main Google Cloud services in this domain?

Start with workload requirements for latency, state, scale, compliance, team capability, recovery and cost. Select the managed service whose operating model fits those requirements and document the consequences in an ADR.

3. What evidence would you require before production approval?

Architecture and threat models, reviewed IaC plan, effective-access results, network and encryption tests, known audit events, SLO and recovery validation, ownership metadata, budget routing and rollback evidence.

4. How do you reduce provider lock-in without creating a lowest-common-denominator platform?

Preserve portable source, data, interfaces and deployment artifacts where a credible exit scenario requires them, while still using provider-specific capabilities when their measurable value exceeds migration and dependency risk.

Chapter Summary

- Zonal and regional workload resilience defines a key architecture boundary.
- Global load balancing and health checks must be automated and least-privileged.
- Cross-region data and backup patterns requires policy, telemetry and recovery evidence.
- Recovery orchestration and failback should be tested before production and after material change.

Chapter 38: Google Cloud Operations Suite and OpenTelemetry

Correlating metrics, logs, traces, profiles and audit data

Why this chapter matters: Operational visibility should follow the user request and service dependency path. Collection, retention, access and cost need the same governance as other data systems.

Learning Objectives

- Explain the architecture, responsibility boundaries and major trade-offs of cloud monitoring metrics and alerting.
- Design cloud logging and centralized sinks for enterprise scale, least privilege and bounded failure.
- Implement cloud trace, profiler and opentelemetry through version-controlled automation and policy checks.
- Troubleshoot audit logs, error reporting and incident investigation using provider-native telemetry and repeatable evidence.
- Deliver a sanitized portfolio artifact for create a google cloud cross-signal incident investigation.

Core Concepts

Cloud Monitoring metrics and alerting

Cloud Monitoring metrics and alerting is a design boundary, not merely a console feature. In Google Cloud Operations Suite and OpenTelemetry, the team must document ownership, trust assumptions, dependency direction, failure impact and the evidence used to prove the control. The recommended implementation uses Cloud Monitoring, Cloud Logging, Cloud Trace / OpenTelemetry, Cloud Profiler only where they satisfy an explicit requirement; equivalent services are not automatically interchangeable.

Cloud Logging and centralized sinks

A production implementation of Cloud Logging and centralized sinks separates the management plane from workload execution and constrains blast radius. Architecture reviews should evaluate quotas, regional behavior, identity propagation, private connectivity, encryption, logging and recovery. The operational contract must state who changes the control, who monitors it and how an emergency exception expires.

Cloud Trace, Profiler and OpenTelemetry

For Cloud Trace, Profiler and OpenTelemetry, repeatability matters more than a one-time successful deployment. Infrastructure as code, policy tests, deployment verification, drift detection and rollback evidence should be retained together. A design is incomplete when it works only for the original engineer or depends on undocumented console steps.

Audit Logs, Error Reporting and incident investigation

The strongest validation for Audit Logs, Error Reporting and incident investigation combines an allowed test, a denied test and a failure test. Teams should prove normal workload behavior, demonstrate that prohibited behavior is blocked and exercise recovery under a bounded fault. Metrics, logs, audit events and cost records become part of the acceptance package.

Service and Capability Map

Capability	Service or pattern	Enterprise use
Metrics	Cloud Monitoring	Collect platform and custom metrics and SLOs.
Logs	Cloud Logging	Aggregate, route, retain and query logs.
Tracing	Cloud Trace / OpenTelemetry	Correlate distributed request latency.
Profiling	Cloud Profiler	Analyze production CPU and memory behavior.
Errors	Error Reporting	Group and alert on application errors.

Capability	Service or pattern	Enterprise use
Audit	Cloud Audit Logs	Investigate administrative and data-access activity.

Architecture and Operating Decisions

- Use Google Cloud managed services only after recording availability, security, portability and cost consequences.
- Separate production, nonproduction, security and shared services with strong administrative and billing boundaries.
- Prefer short-lived workload identity, private service access and centralized audit evidence over persistent credentials and public administration.
- Automate cloud trace, profiler and opentelemetry with reviewed plans, policy tests, staged rollout and post-deployment verification.
- Define recovery objectives, quotas, ownership and decommissioning before the service becomes business critical.

Reference Configuration or Decision Pattern

```
observability:
  telemetry: [metrics, structured_logs, traces, profiles, audit]
  routing: aggregated_sinks_by_data_class
  application: opentelemetry
  alerts: user_impact_and_slo_burn
  retention: explicit_per_log_bucket
  access: least_privilege_with_separate_security_evidence
```

Interpretation: Treat the pattern as a starting point. Replace placeholders with measured requirements, provider limits and approved organizational standards.

Production Readiness Checklist

- ☐ Architecture diagrams and responsibility boundaries for google cloud operations suite and opentelemetry are current and version controlled.
- ☐ Identity uses groups, roles, workload federation and privileged elevation with no unnecessary standing administrator access.
- ☐ Network paths, DNS behavior, encryption, audit logging and service limits are tested before production use.
- ☐ Changes are delivered through IaC with validation, policy, approval, rollback and drift detection.
- ☐ Recovery, cost allocation, ownership metadata and teardown have measurable evidence.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment succeeds but service is unreachable	Route, firewall, DNS or private-endpoint path is incomplete	Trace the path hop by hop; compare route, policy, DNS and endpoint state; then run an explicit reachability test.	Successful synthetic request and flow/audit evidence
Authorization is broader than intended	Inherited role, wildcard permission or unmanaged credential	Inspect effective permissions, credential age and access logs; replace broad grants with scoped roles and short-lived identity.	Effective-access review and denied-action test
Regional failure breaks the workload	State, dependency or failover mechanism remained single-region	Map stateful dependencies, restore order and traffic controls; exercise failover and fallback against measured RTO/RPO.	Timed recovery log with data-integrity checks
Cost rises without traffic growth	Idle capacity, data transfer, logging volume or commitment mismatch	Break down spend by owner, service, region and usage driver; validate rightsizing or commitment changes against SLOs.	Cost delta plus performance and reliability validation

Hands-on Lab 38: Create a Google Cloud Cross-Signal Incident Investigation

Prerequisites

- Disposable provider sandbox or a local simulation.
- Terraform or provider CLI, Git and a diagramming tool.
- Synthetic identities and data; no production credentials or confidential information.

Implementation

1. Create the architecture and responsibility model for google cloud operations suite and opentelemetry using fictional enterprise requirements.
2. Implement the smallest safe sandbox using Cloud Monitoring, Cloud Logging, Cloud Trace / OpenTelemetry; use free or low-cost options where available.
3. Apply identity, network, encryption, logging, metadata and budget controls through IaC or an equivalent version-controlled simulation.
4. Run one approved behavior test and one denied behavior test; save the commands, timestamps and resulting telemetry.
5. Inject a bounded configuration or dependency failure, follow the runbook, recover and record measured recovery time.
6. Destroy or quarantine the lab, revoke temporary access and verify that no unexpected billable resources remain.

Validation Evidence

- Version-controlled architecture, IaC and decision record.
- Allowed, denied and failure-test outputs with timestamps.
- Cost, recovery, ownership and teardown verification.

Cleanup

- Destroy test resources in dependency order and verify the provider inventory and billing view.
- Revoke temporary identities, remove local state copies and sanitize all portfolio evidence.

Interview Questions and Model Answers

1. What is the most important design principle in Google Cloud Operations Suite and OpenTelemetry?

Make responsibility and failure boundaries explicit, then automate controls and prove them with normal, denied and recovery tests rather than trusting a successful deployment alone.

2. How would you choose between the main Google Cloud services in this domain?

Start with workload requirements for latency, state, scale, compliance, team capability, recovery and cost. Select the managed service whose operating model fits those requirements and document the consequences in an ADR.

3. What evidence would you require before production approval?

Architecture and threat models, reviewed IaC plan, effective-access results, network and encryption tests, known audit events, SLO and recovery validation, ownership metadata, budget routing and rollback evidence.

4. How do you reduce provider lock-in without creating a lowest-common-denominator platform?

Preserve portable source, data, interfaces and deployment artifacts where a credible exit scenario requires them, while still using provider-specific capabilities when their measurable value exceeds migration and dependency risk.

Chapter Summary

- Cloud Monitoring metrics and alerting defines a key architecture boundary.
- Cloud Logging and centralized sinks must be automated and least-privileged.
- Cloud Trace, Profiler and OpenTelemetry requires policy, telemetry and recovery evidence.
- Audit Logs, Error Reporting and incident investigation should be tested before production and after material change.

Chapter 39: Google Cloud FinOps, Billing Exports and Resource Governance

Connecting project ownership and usage to business value

Why this chapter matters: Google Cloud cost engineering requires billing export, hierarchy, labels or tags, commitments and workload-level unit metrics. Optimization decisions must include performance and reliability.

Learning Objectives

- Explain the architecture, responsibility boundaries and major trade-offs of cloud billing export and cost allocation.
- Design budgets, quotas and anomaly operations for enterprise scale, least privilege and bounded failure.
- Implement committed use discounts and rightsizing through version-controlled automation and policy checks.
- Troubleshoot finops hub, active assist and unit economics using provider-native telemetry and repeatable evidence.
- Deliver a sanitized portfolio artifact for build a google cloud billing and unit-cost dashboard.

Core Concepts

Cloud Billing export and cost allocation

Cloud Billing export and cost allocation is a design boundary, not merely a console feature. In Google Cloud FinOps, Billing Exports and Resource Governance, the team must document ownership, trust assumptions, dependency direction, failure impact and the evidence used to prove the control. The recommended implementation uses Cloud Billing export to BigQuery, Budgets and alerts, Active Assist / Recommender, Committed use discounts only where they satisfy an explicit requirement; equivalent services are not automatically interchangeable.

Budgets, quotas and anomaly operations

A production implementation of Budgets, quotas and anomaly operations separates the management plane from workload execution and constrains blast radius. Architecture reviews should evaluate quotas, regional behavior, identity propagation, private connectivity, encryption, logging and recovery. The operational contract must state who changes the control, who monitors it and how an emergency exception expires.

Committed use discounts and rightsizing

For Committed use discounts and rightsizing, repeatability matters more than a one-time successful deployment. Infrastructure as code, policy tests, deployment verification, drift detection and rollback evidence should be retained together. A design is incomplete when it works only for the original engineer or depends on undocumented console steps.

FinOps Hub, Active Assist and unit economics

The strongest validation for FinOps Hub, Active Assist and unit economics combines an allowed test, a denied test and a failure test. Teams should prove normal workload behavior, demonstrate that prohibited behavior is blocked and exercise recovery under a bounded fault. Metrics, logs, audit events and cost records become part of the acceptance package.

Service and Capability Map

Capability	Service or pattern	Enterprise use
Billing data	Cloud Billing export to BigQuery	Analyze detailed cost and usage.
Control	Budgets and alerts	Route forecast and threshold notifications.
Optimization	Active Assist / Recommender	Identify rightsizing and idle-resource opportunities.

Capability	Service or pattern	Enterprise use
Commitments	Committed use discounts	Discount stable eligible usage.
Governance	Tags / labels / project hierarchy	Allocate ownership and apply policy.
FinOps	FinOps Hub	Centralize optimization and FinOps insights.

Architecture and Operating Decisions

- Use Google Cloud managed services only after recording availability, security, portability and cost consequences.
- Separate production, nonproduction, security and shared services with strong administrative and billing boundaries.
- Prefer short-lived workload identity, private service access and centralized audit evidence over persistent credentials and public administration.
- Automate committed use discounts and rightsizing with reviewed plans, policy tests, staged rollout and post-deployment verification.
- Define recovery objectives, quotas, ownership and decommissioning before the service becomes business critical.

Reference Configuration or Decision Pattern

```
finops:
  export: detailed_billing_to_bigquery
  allocation: project_plus_tags
  budgets: owner_and_product
  recommendations: reviewed_with_slo_context
  commitments: conservative_baseline
  unit_metrics: [cost_per_request, cost_per_dataset_processed]
  cadence: weekly_operations_monthly_commitments
```

Interpretation: Treat the pattern as a starting point. Replace placeholders with measured requirements, provider limits and approved organizational standards.

Production Readiness Checklist

- ☐ Architecture diagrams and responsibility boundaries for google cloud finops, billing exports and resource governance are current and version controlled.
- ☐ Identity uses groups, roles, workload federation and privileged elevation with no unnecessary standing administrator access.
- ☐ Network paths, DNS behavior, encryption, audit logging and service limits are tested before production use.
- ☐ Changes are delivered through IaC with validation, policy, approval, rollback and drift detection.
- ☐ Recovery, cost allocation, ownership metadata and teardown have measurable evidence.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment succeeds but service is unreachable	Route, firewall, DNS or private-endpoint path is incomplete	Trace the path hop by hop; compare route, policy, DNS and endpoint state; then run an explicit reachability test.	Successful synthetic request and flow/audit evidence
Authorization is broader than intended	Inherited role, wildcard permission or unmanaged credential	Inspect effective permissions, credential age and access logs; replace broad grants with scoped roles and short-lived identity.	Effective-access review and denied-action test
Regional failure breaks the workload	State, dependency or failover mechanism remained single-region	Map stateful dependencies, restore order and traffic controls; exercise failover and fallback against measured RTO/RPO.	Timed recovery log with data-integrity checks
Cost rises without traffic growth	Idle capacity, data transfer, logging volume or commitment mismatch	Break down spend by owner, service, region and usage driver; validate rightsizing or commitment changes against SLOs.	Cost delta plus performance and reliability validation

Hands-on Lab 39: Build a Google Cloud Billing and Unit-Cost Dashboard

Prerequisites

- Disposable provider sandbox or a local simulation.
- Terraform or provider CLI, Git and a diagramming tool.
- Synthetic identities and data; no production credentials or confidential information.

Implementation

1. Create the architecture and responsibility model for google cloud finops, billing exports and resource governance using fictional enterprise requirements.
2. Implement the smallest safe sandbox using Cloud Billing export to BigQuery, Budgets and alerts, Active Assist / Recommender; use free or low-cost options where available.
3. Apply identity, network, encryption, logging, metadata and budget controls through IaC or an equivalent version-controlled simulation.
4. Run one approved behavior test and one denied behavior test; save the commands, timestamps and resulting telemetry.
5. Inject a bounded configuration or dependency failure, follow the runbook, recover and record measured recovery time.
6. Destroy or quarantine the lab, revoke temporary access and verify that no unexpected billable resources remain.

Validation Evidence

- Version-controlled architecture, IaC and decision record.
- Allowed, denied and failure-test outputs with timestamps.
- Cost, recovery, ownership and teardown verification.

Cleanup

- Destroy test resources in dependency order and verify the provider inventory and billing view.
- Revoke temporary identities, remove local state copies and sanitize all portfolio evidence.

Interview Questions and Model Answers

1. What is the most important design principle in Google Cloud FinOps, Billing Exports and Resource Governance?

Make responsibility and failure boundaries explicit, then automate controls and prove them with normal, denied and recovery tests rather than trusting a successful deployment alone.

2. How would you choose between the main Google Cloud services in this domain?

Start with workload requirements for latency, state, scale, compliance, team capability, recovery and cost. Select the managed service whose operating model fits those requirements and document the consequences in an ADR.

3. What evidence would you require before production approval?

Architecture and threat models, reviewed IaC plan, effective-access results, network and encryption tests, known audit events, SLO and recovery validation, ownership metadata, budget routing and rollback evidence.

4. How do you reduce provider lock-in without creating a lowest-common-denominator platform?

Preserve portable source, data, interfaces and deployment artifacts where a credible exit scenario requires them, while still using provider-specific capabilities when their measurable value exceeds migration and dependency risk.

Chapter Summary

- Cloud Billing export and cost allocation defines a key architecture boundary.
- Budgets, quotas and anomaly operations must be automated and least-privileged.
- Committed use discounts and rightsizing requires policy, telemetry and recovery evidence.
- FinOps Hub, Active Assist and unit economics should be tested before production and after material change.

Chapter 40: Google Cloud Enterprise Foundation Capstone

Delivering a governed project from request to recovery and closure

Why this chapter matters: The Google Cloud foundation is complete when project vending, Shared VPC, IAM, policy, security, telemetry, cost and recovery operate as one tested platform product.

Learning Objectives

- Explain the architecture, responsibility boundaries and major trade-offs of project factory and organization policy.
- Design shared vpc and private connectivity for enterprise scale, least privilege and bounded failure.
- Implement workload deployment and recovery through version-controlled automation and policy checks.
- Troubleshoot security, operations, cost and lifecycle evidence using provider-native telemetry and repeatable evidence.
- Deliver a sanitized portfolio artifact for deliver the google cloud enterprise foundation capstone.

Core Concepts

Project factory and organization policy

Project factory and organization policy is a design boundary, not merely a console feature. In Google Cloud Enterprise Foundation Capstone, the team must document ownership, trust assumptions, dependency direction, failure impact and the evidence used to prove the control. The recommended implementation uses Organization / folders / projects, IAM / federation / workload identity, Shared VPC / Interconnect / PSC, SCC / VPC SC / KMS only where they satisfy an explicit requirement; equivalent services are not automatically interchangeable.

Shared VPC and private connectivity

A production implementation of Shared VPC and private connectivity separates the management plane from workload execution and constrains blast radius. Architecture reviews should evaluate quotas, regional behavior, identity propagation, private connectivity, encryption, logging and recovery. The operational contract must state who changes the control, who monitors it and how an emergency exception expires.

Workload deployment and recovery

For Workload deployment and recovery, repeatability matters more than a one-time successful deployment. Infrastructure as code, policy tests, deployment verification, drift detection and rollback evidence should be retained together. A design is incomplete when it works only for the original engineer or depends on undocumented console steps.

Security, operations, cost and lifecycle evidence

The strongest validation for Security, operations, cost and lifecycle evidence combines an allowed test, a denied test and a failure test. Teams should prove normal workload behavior, demonstrate that prohibited behavior is blocked and exercise recovery under a bounded fault. Metrics, logs, audit events and cost records become part of the acceptance package.

Service and Capability Map

Capability	Service or pattern	Enterprise use
Hierarchy	Organization / folders / projects	Provide governance and workload boundaries.
Identity	IAM / federation / workload identity	Control workforce and software access.
Network	Shared VPC / Interconnect / PSC	Provide governed connectivity.
Security	SCC / VPC SC / KMS	Protect configuration and sensitive data.
Operations	Cloud Operations suite	Monitor and investigate workloads.

Capability	Service or pattern	Enterprise use
Cost	Billing export / budgets	Allocate and govern spend.

Architecture and Operating Decisions

- Use Google Cloud managed services only after recording availability, security, portability and cost consequences.
- Separate production, nonproduction, security and shared services with strong administrative and billing boundaries.
- Prefer short-lived workload identity, private service access and centralized audit evidence over persistent credentials and public administration.
- Automate workload deployment and recovery with reviewed plans, policy tests, staged rollout and post-deployment verification.
- Define recovery objectives, quotas, ownership and decommissioning before the service becomes business critical.

Reference Configuration or Decision Pattern

```
acceptance:
  project_vending_minutes: 30
  service_account_keys: 0
  public_storage_prevention: enforced
  central_audit_visibility_minutes: 5
  regional_recovery_tested: true
  allocation_coverage_percent: 100
  teardown_residual_billable_resources: 0
```

Interpretation: Treat the pattern as a starting point. Replace placeholders with measured requirements, provider limits and approved organizational standards.

Production Readiness Checklist

- ☐ Architecture diagrams and responsibility boundaries for google cloud enterprise foundation capstone are current and version controlled.
- ☐ Identity uses groups, roles, workload federation and privileged elevation with no unnecessary standing administrator access.
- ☐ Network paths, DNS behavior, encryption, audit logging and service limits are tested before production use.
- ☐ Changes are delivered through IaC with validation, policy, approval, rollback and drift detection.
- ☐ Recovery, cost allocation, ownership metadata and teardown have measurable evidence.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment succeeds but service is unreachable	Route, firewall, DNS or private-endpoint path is incomplete	Trace the path hop by hop; compare route, policy, DNS and endpoint state; then run an explicit reachability test.	Successful synthetic request and flow/audit evidence
Authorization is broader than intended	Inherited role, wildcard permission or unmanaged credential	Inspect effective permissions, credential age and access logs; replace broad grants with scoped roles and short-lived identity.	Effective-access review and denied-action test
Regional failure breaks the workload	State, dependency or failover mechanism remained single-region	Map stateful dependencies, restore order and traffic controls; exercise failover and fallback against measured RTO/RPO.	Timed recovery log with data-integrity checks
Cost rises without traffic growth	Idle capacity, data transfer, logging volume or commitment mismatch	Break down spend by owner, service, region and usage driver; validate rightsizing or commitment changes against SLOs.	Cost delta plus performance and reliability validation

Hands-on Lab 40: Deliver the Google Cloud Enterprise Foundation Capstone

Prerequisites

- Disposable provider sandbox or a local simulation.
- Terraform or provider CLI, Git and a diagramming tool.
- Synthetic identities and data; no production credentials or confidential information.

Implementation

1. Create the architecture and responsibility model for google cloud enterprise foundation capstone using fictional enterprise requirements.
2. Implement the smallest safe sandbox using Organization / folders / projects, IAM / federation / workload identity, Shared VPC / Interconnect / PSC; use free or low-cost options where available.
3. Apply identity, network, encryption, logging, metadata and budget controls through IaC or an equivalent version-controlled simulation.
4. Run one approved behavior test and one denied behavior test; save the commands, timestamps and resulting telemetry.
5. Inject a bounded configuration or dependency failure, follow the runbook, recover and record measured recovery time.
6. Destroy or quarantine the lab, revoke temporary access and verify that no unexpected billable resources remain.

Validation Evidence

- Version-controlled architecture, IaC and decision record.
- Allowed, denied and failure-test outputs with timestamps.
- Cost, recovery, ownership and teardown verification.

Cleanup

- Destroy test resources in dependency order and verify the provider inventory and billing view.
- Revoke temporary identities, remove local state copies and sanitize all portfolio evidence.

Interview Questions and Model Answers

1. What is the most important design principle in Google Cloud Enterprise Foundation Capstone?

Make responsibility and failure boundaries explicit, then automate controls and prove them with normal, denied and recovery tests rather than trusting a successful deployment alone.

2. How would you choose between the main Google Cloud services in this domain?

Start with workload requirements for latency, state, scale, compliance, team capability, recovery and cost. Select the managed service whose operating model fits those requirements and document the consequences in an ADR.

3. What evidence would you require before production approval?

Architecture and threat models, reviewed IaC plan, effective-access results, network and encryption tests, known audit events, SLO and recovery validation, ownership metadata, budget routing and rollback evidence.

4. How do you reduce provider lock-in without creating a lowest-common-denominator platform?

Preserve portable source, data, interfaces and deployment artifacts where a credible exit scenario requires them, while still using provider-specific capabilities when their measurable value exceeds migration and dependency risk.

Chapter Summary

- Project factory and organization policy defines a key architecture boundary.
- Shared VPC and private connectivity must be automated and least-privileged.
- Workload deployment and recovery requires policy, telemetry and recovery evidence.
- Security, operations, cost and lifecycle evidence should be tested before production and after material change.

Google Cloud Enterprise Architecture and Operations Reference

Use this reference to review architecture outcomes, implementation controls and the evidence required for production approval.

Domain	Required outcome	Validation evidence
Hierarchy and ownership	Durable resource boundaries with accountable owners	Inventory, effective policy and lifecycle tests
Identity	Federated humans and keyless workloads	Token, role, access-review and denied-action evidence
Network	Approved routes, DNS, private access and egress	Reachability, flow, DNS, latency and failover tests
Data	Classified, encrypted and recoverable state	Backup, restore, reconciliation and exit evidence
Security	Preventive, detective and recovery controls	Policy results, findings, containment and audit timeline
Operations	SLOs, telemetry, runbooks and incident process	Dashboards, alerts, drill timeline and postmortem
FinOps	Allocation, budgets, unit economics and commitment governance	Billing export, owner coverage and optimization record
Delivery	Reviewed IaC and immutable artifact promotion	Plan, policy, signature, deployment and rollback chain
Reliability	Measured capacity, RTO/RPO and failback	Load, failure, recovery and integrity tests
Lifecycle	Vending, upgrade, exception and closure	Fresh deployment, drift and zero-residual teardown

Completion Checklist

- ☐ I can explain the architecture and trade-offs in every chapter of 12D.
- ☐ I can implement at least one sandbox pattern through IaC and short-lived identity.
- ☐ I can demonstrate approved, denied and failure behavior with provider-native evidence.
- ☐ I can design private connectivity, data protection, recovery and cost governance.
- ☐ I can produce architecture diagrams, ADRs, runbooks and troubleshooting timelines.
- ☐ I can destroy the lab safely and prove that no unexpected billable resource remains.

Additional Portfolio Projects

1. Resource vending workflow with ownership, policy, budget and closure automation.
2. Keyless CI/CD identity module and effective-access test harness.
3. Private network and DNS validation pack with route and failure scenarios.
4. Managed data backup, restore and reconciliation project.
5. Security finding triage and automated containment simulation.
6. SLO dashboard and incident-command game day.
7. Billing export, allocation coverage and unit-cost dashboard.
8. IaC provider-upgrade and drift-remediation pipeline.
9. Regional or cross-provider disaster-recovery exercise.
10. Executive architecture brief with decisions, risks, roadmap and measured outcomes.

30-Day Practice Plan

Days	Focus	Required output
1-4	Hierarchy, ownership and identity	Architecture diagram, RACI and access tests
5-8	Networking and private services	IP plan, flow matrix and reachability evidence
9-12	Compute and delivery	Workload deployment, autoscaling and rollback
13-16	Data and recovery	Backup, restore and integrity report
17-20	Security and audit	Threat model, policy tests and finding workflow
21-23	Observability and incidents	SLO, dashboard and incident timeline
24-26	FinOps and governance	Allocation, budget and unit-cost review
27-28	Failure drills	Bounded outage and recovery evidence
29-30	Capstone and portfolio	Architecture defense, cleanup and sanitized report

Glossary

Term	Definition
Application landing zone	A governed environment in which a workload team deploys and operates applications.
Blast radius	The maximum scope of impact from a failure, credential or change.
Control plane	Interfaces and services used to administer a cloud or platform.
Data plane	The runtime path that processes application traffic and data.
Federation	Trust that allows an external identity source to authenticate without duplicate long-lived credentials.
Guardrail	A preventive, detective or corrective control that constrains activity.
Landing zone	A scalable cloud foundation for hierarchy, identity, network, security, logging and cost.
Private endpoint	A private network interface used to access a managed service without a public path.
RPO	Maximum acceptable data loss measured in time.
RTO	Target time to restore a service after disruption.
Unit cost	Technology cost per useful business or technical output.
Workload identity	A short-lived identity issued to software based on trusted runtime context.

Official Documentation Index

The following official sources were checked for this 2026 edition. Verify current service behavior, quotas, prices and certification requirements before production use or exam registration.

- [Google Cloud Landing Zone Design](#)
- [Enterprise Foundations Blueprint](#)
- [Google Cloud Well-Architected Framework](#)
- [Cloud IAM Documentation](#)
- [Shared VPC Overview](#)
- [Network Connectivity Center](#)
- [Security Command Center](#)
- [Cloud Billing Export](#)

Volume 12D completion: preserve your architecture, code, test evidence, recovery timeline, cost review and sanitized final report as one portfolio package.